

Data Vortex A'26 — Round 3

Public Reaction to Changes in the Recommendation Algorithm

Team: hy4366 \n**Source:** Hacker News public Algolia archive API \n**Important limitation:** This is a historical public archive sample collected through the Hacker News Algolia archive API; it is not a continuous live stream.

```
In [1]: from pathlib import Path
import json, sys, pandas as pd, matplotlib.pyplot as plt
from IPython.display import display, Image
ROOT = Path.cwd().resolve(); ROOT = ROOT.parent if ROOT.name == 'notebooks' else RO
sys.path.insert(0, str(ROOT/'src'))
metadata = json.loads((ROOT/'data/raw/collection_metadata.json').read_text(encoding
print('Collection (UTC):', metadata['collection_start_timestamp'], 'to', metadata['
print('Raw archive source:', metadata['source'])
print('Queries:', '; '.join(metadata['query_terms']))
```

Collection (UTC): 2026-09-21T19:44:34.808161+00:00 to 2026-09-21T19:44:49.802497+00:00

Raw archive source: Hacker News public Algolia archive API

Queries: Twitter recommendation algorithm; YouTube recommendation algorithm; Instagram algorithm change; feed algorithm; recommendation algorithm; recommendation ranking; Facebook news feed algorithm; algorithmic feed

1. Dataset loading and data-quality audit

The correction pass reuses the saved raw archive export; it does not collect new data.

```
In [2]: raw = pd.read_csv(ROOT/'data/raw/hn_recommendation_algorithm_reactions_raw.csv')
df = pd.read_csv(ROOT/'data/round3_recommendation_algorithm_reactions.csv')
raw['timestamp'] = pd.to_datetime(raw['timestamp'], utc=True)
df['timestamp'] = pd.to_datetime(df['timestamp'], utc=True)
audit = pd.read_csv(ROOT/'results/relevance_filter_audit.csv')
print('Raw records:', len(raw), '| final retained:', len(df))
print('Observation window:', df.timestamp.min(), 'to', df.timestamp.max())
display(audit)
display(pd.DataFrame({'missing_values': df.isna().sum(), 'dtype': df.dtypes.astype(
print('Exact duplicate rows:', df.duplicated().sum(), '| duplicate cleaned text:',
display(df[['source', 'source_type', 'query_term']].value_counts().to_frame('records'
```

Raw records: 389 | final retained: 204

Observation window: 2011-02-02 06:34:14+00:00 to 2026-09-21 18:49:57+00:00

initial_screened_records **excluded_empty_or_short_text** **excluded_not_specific_to_change_or_i**

0

389

0



	missing_values	dtype
id	0	str
source	0	str
source_type	0	str
timestamp	0	datetime64[us, UTC]
date	0	str
text	0	str
title	0	str
url	0	str
author_or_author_id_if_appropriate	0	str
engagement_score	152	float64
comment_count	152	float64
upvote_count_if_available	152	float64
query_term	0	str
topic_relevance	0	str
story_id	0	int64
story_url	72	str
collected_at	0	str
collection_timezone	0	str
cleaned_text	0	str
relevance_filter_reason	0	str
predicted_sentiment	0	str
negative_probability	0	float64
neutral_probability	0	float64
positive_probability	0	float64

Exact duplicate rows: 0 | duplicate cleaned text: 0 | empty text: 0

			records
source	source_type	query_term	
Hacker News	public_comment	Twitter recommendation algorithm	35
		YouTube recommendation algorithm	33
	public_story	feed algorithm	26
		Instagram algorithm change	25
	public_comment	recommendation ranking	24
		Facebook news feed algorithm	20
		algorithmic feed	15
		Twitter recommendation algorithm	11
	public_story	YouTube recommendation algorithm	7
		recommendation algorithm	6
		recommendation ranking	1
		Facebook news feed algorithm	1

2. Preprocessing and Round 2 NLP model

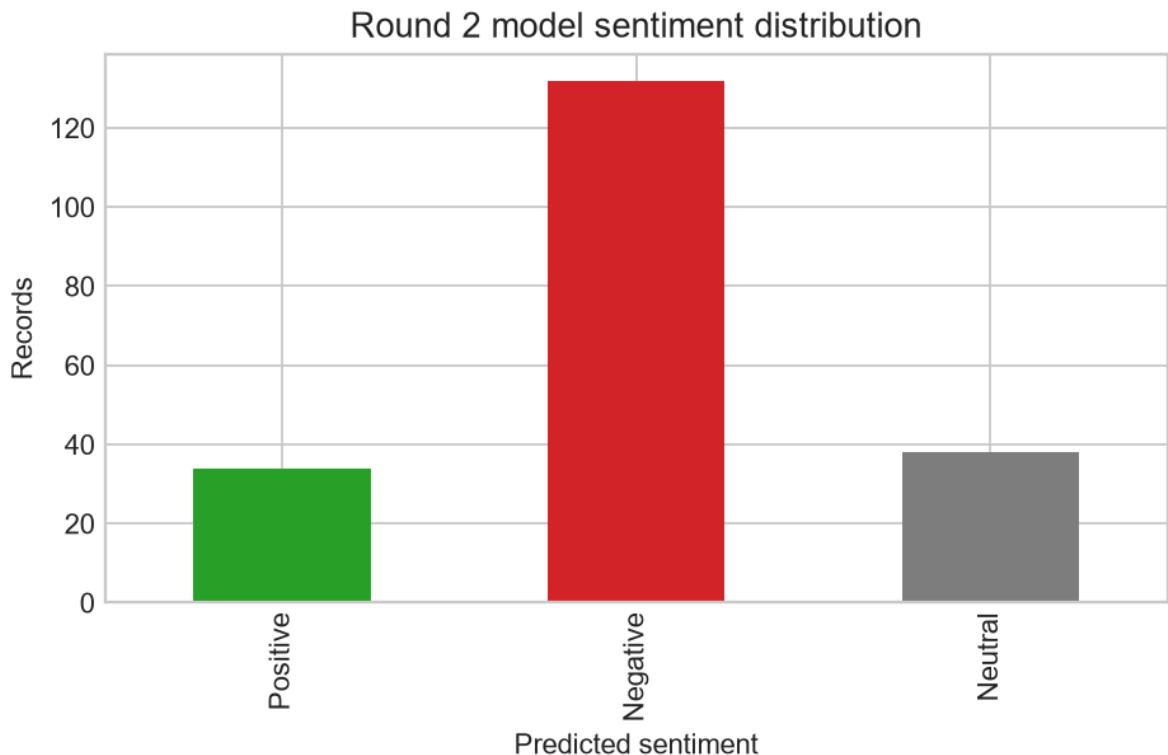
Raw text is preserved. The documented stricter relevance screen requires recommendation/feed context plus a concrete change, reaction, quality, or user-control signal. The exact preserved Round 2 TF-IDF + Logistic Regression pipeline is loaded directly.

```
In [3]: import joblib
model = joblib.load(ROOT/'models/sentiment_label_pipeline.pkl')
print('Model type:', type(model))
print('Classes:', model.classes_)
check = model.predict(df.cleaned_text)
print('Predictions reproduce saved CSV:', bool((check == df.predicted_sentiment).all))
counts = df.predicted_sentiment.value_counts().reindex(['Positive', 'Negative', 'Neutral'])
display(pd.DataFrame({'count': counts, 'percent': (counts/len(df)*100).round(2)}))
Image(filename=str(ROOT/'figures/sentiment_distribution.png'))
```

```
Model type: <class 'sklearn.pipeline.Pipeline'>
Classes: ['Negative' 'Neutral' 'Positive']
Predictions reproduce saved CSV: True
```

	count	percent
predicted_sentiment		
Positive	34	16.67
Negative	132	64.71
Neutral	38	18.63

Out[3]:



3. Sentiment over time and descriptive candidate shifts

Monthly index = $(\text{Positive} - \text{Negative}) / (\text{Positive} + \text{Negative} + \text{Neutral})$.

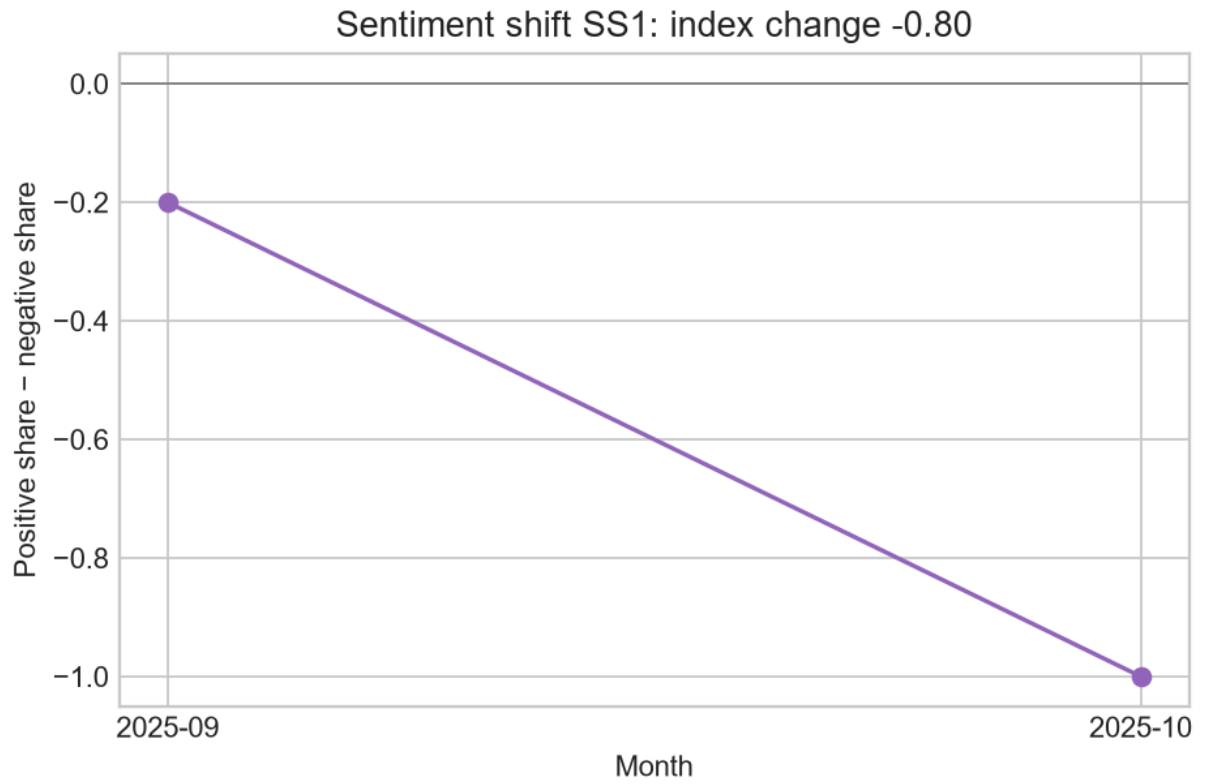
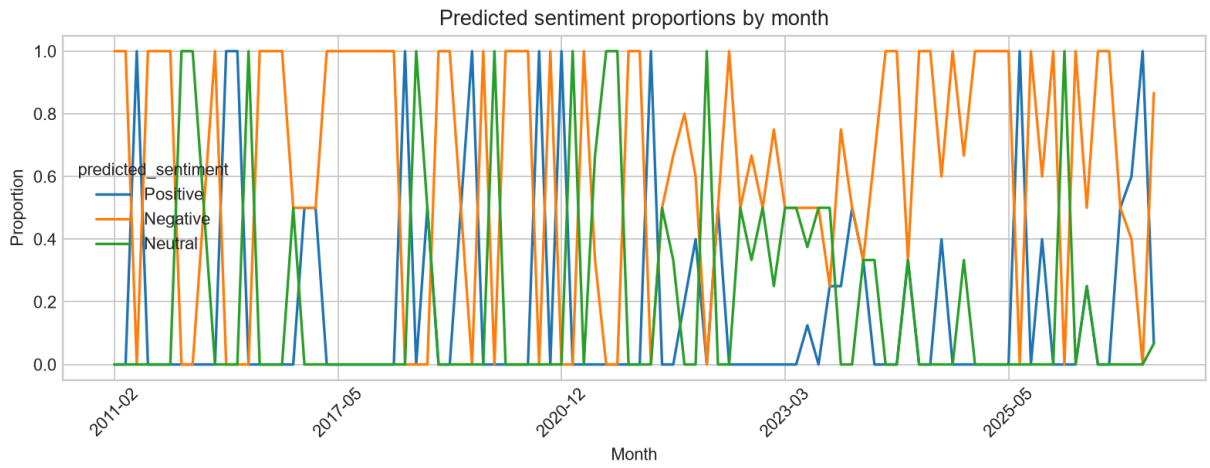
Candidate shifts require at least three records in each adjacent observed month. Because monthly n is small, these are descriptive candidates, not statistically significant behavioural shifts.

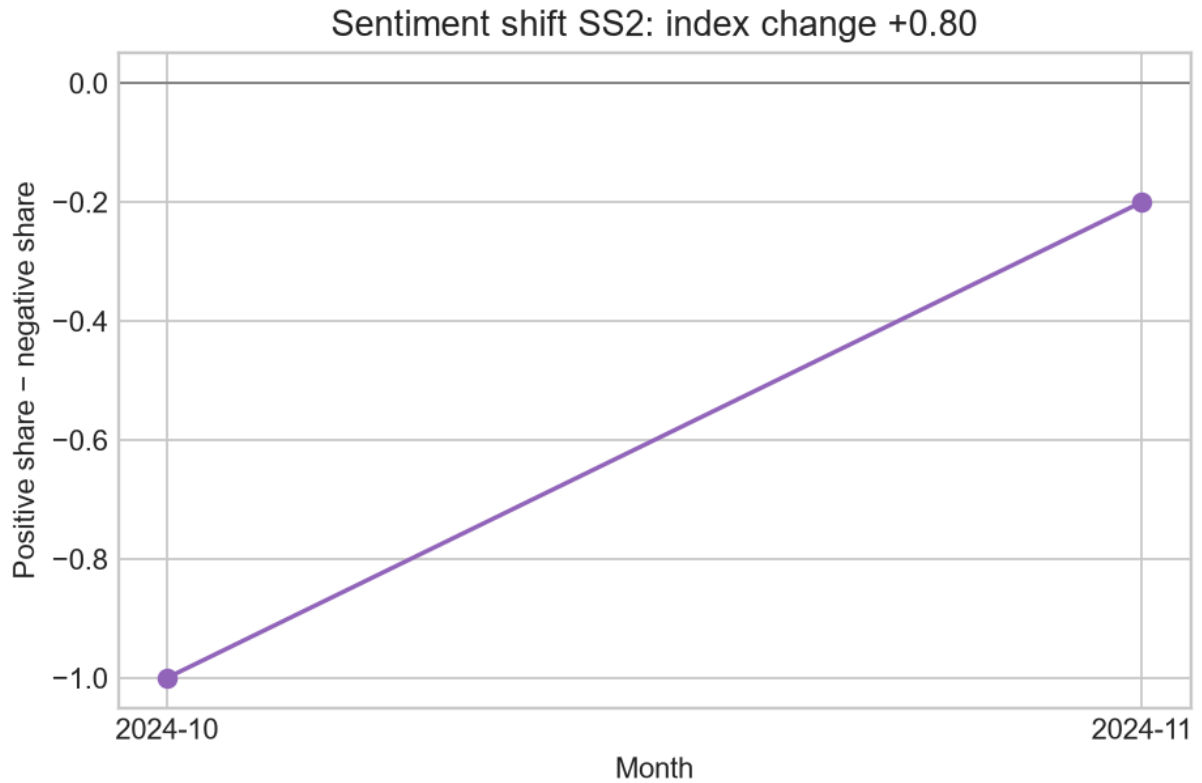
```
In [4]: monthly = pd.read_csv(ROOT/'results/sentiment_by_month.csv')
shifts = pd.read_csv(ROOT/'results/detected_sentiment_shifts.csv')
display(monthly[['month', 'Positive', 'Negative', 'Neutral', 'record_count', 'sentiment_
display(shifts[['shift_id', 'start_time', 'end_time', 'before_sentiment', 'after_sentim
for f in ['sentiment_over_time.png', 'sentiment_shift_1.png', 'sentiment_shift_2.png'
display(Image(filename=str(ROOT/'figures'/f)))
```

	month	Positive	Negative	Neutral	record_count	sentiment_index
74	2024-11	2	3	0	5	-0.200000
75	2024-12	0	3	0	3	-1.000000
76	2025-01	0	2	1	3	-0.666667
77	2025-02	0	3	0	3	-1.000000
78	2025-03	0	1	0	1	-1.000000
79	2025-04	0	3	0	3	-1.000000
80	2025-05	0	1	0	1	-1.000000
81	2025-07	1	0	0	1	1.000000
82	2025-08	0	1	0	1	-1.000000
83	2025-09	2	3	0	5	-0.200000
84	2025-10	0	3	0	3	-1.000000
85	2025-11	0	0	1	1	0.000000
86	2025-12	0	1	0	1	-1.000000
87	2026-01	1	2	1	4	-0.250000
88	2026-02	0	4	0	4	-1.000000
89	2026-03	0	2	0	2	-1.000000
90	2026-05	1	1	0	2	0.000000
91	2026-06	3	2	0	5	0.200000
92	2026-08	1	0	0	1	1.000000
93	2026-09	1	13	1	15	-0.800000

	shift_id	start_time	end_time	before_sentiment	after_sentiment	magnitude	record_count
0	SS1	2025-09	2025-10	-0.2	-1.0	-0.8	
1	SS2	2024-10	2024-11	-1.0	-0.2	0.8	







4. Activity and available Hacker News score

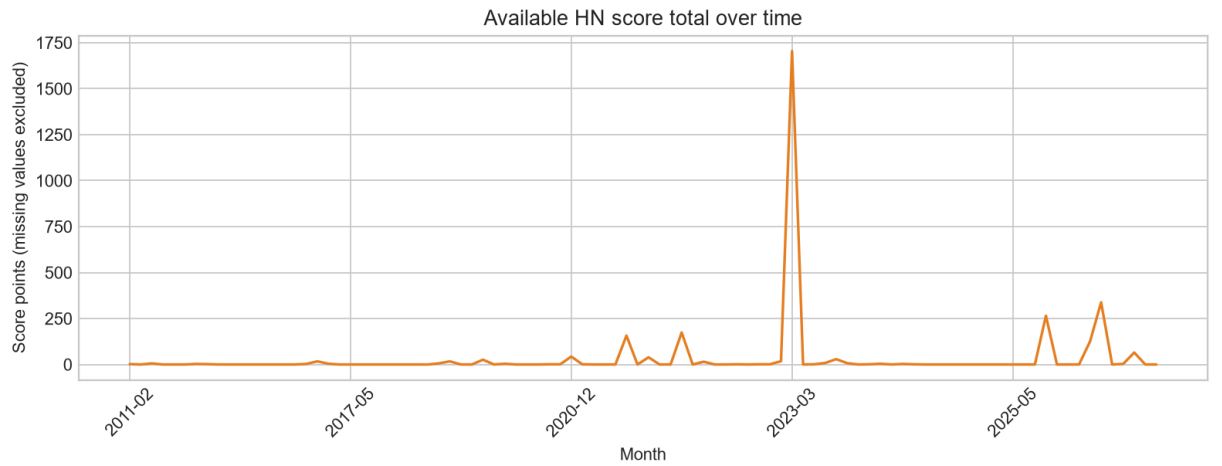
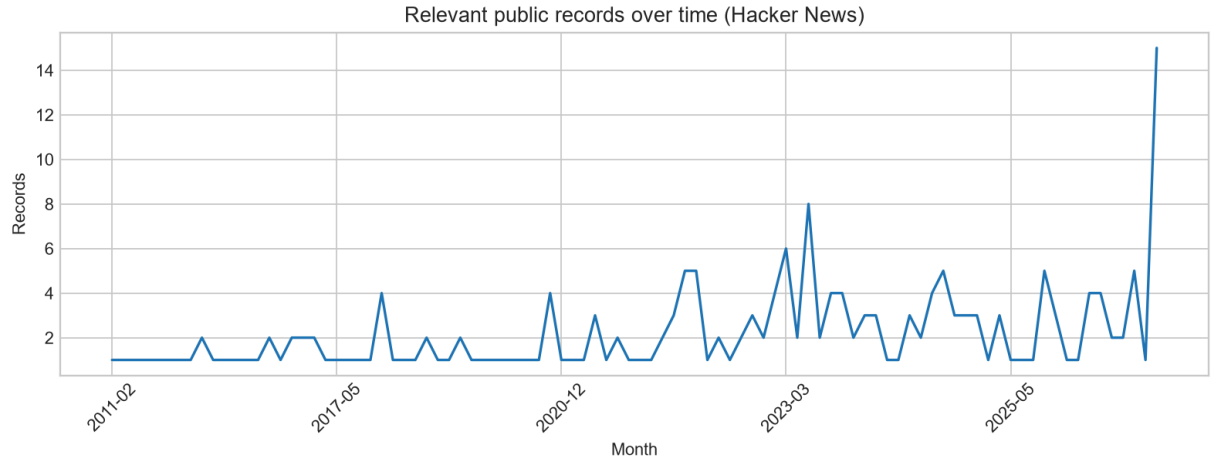
“Engagement” means available Hacker News story score, not a universal social-media engagement measure. The reference baseline is the median monthly total among observed scored months; missing comment scores are not converted to zero.

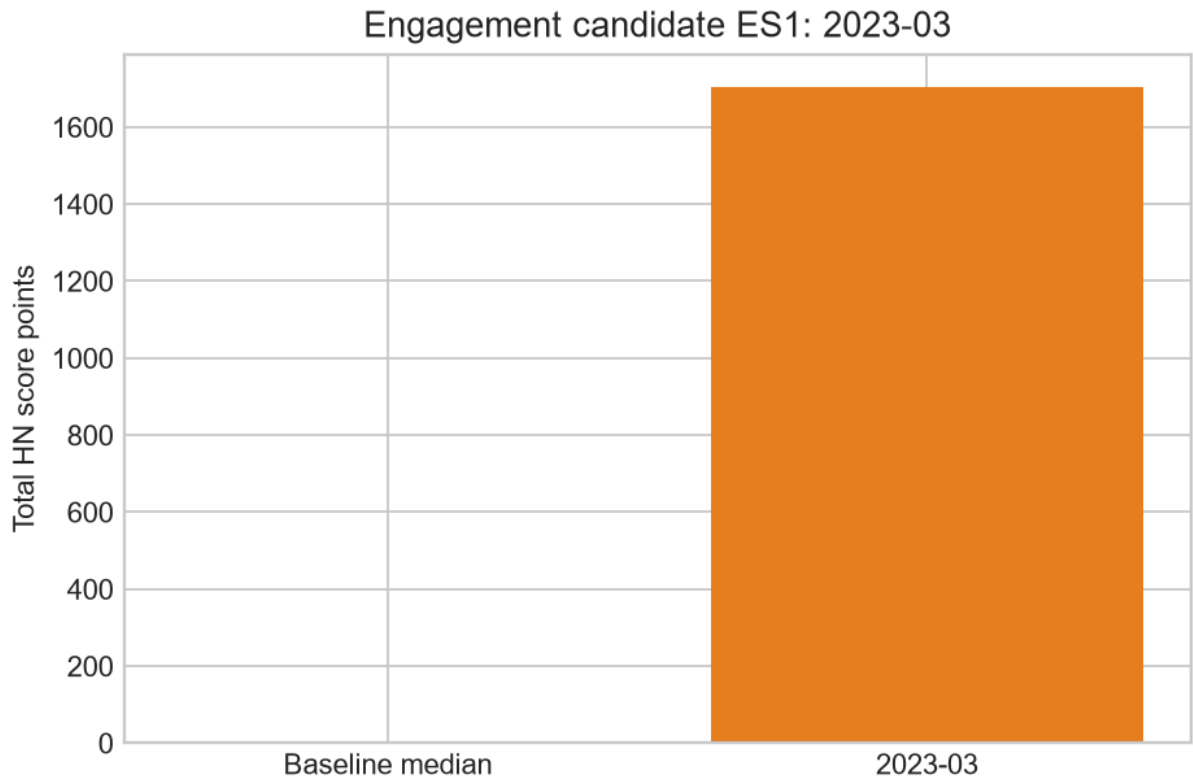
```
In [5]: activity = pd.read_csv(ROOT/'results/activity_results.csv')
spikes = pd.read_csv(ROOT/'results/detected_engagement_spikes.csv')
display(activity.tail(20))
display(spikes[['spike_id', 'timestamp_time_window', 'activity_count', 'engagement_val
for f in ['data_collection_overview.png', 'engagement_over_time.png', 'engagement_spi
display(Image(filename=str(ROOT/'figures'/f)))
```

	month	activity_count	score_record_count	total_engagement	average_engagement	med
74	2024-11	5	0	0.0	NaN	
75	2024-12	3	0	0.0	NaN	
76	2025-01	3	0	0.0	NaN	
77	2025-02	3	0	0.0	NaN	
78	2025-03	1	0	0.0	NaN	
79	2025-04	3	0	0.0	NaN	
80	2025-05	1	0	0.0	NaN	
81	2025-07	1	0	0.0	NaN	
82	2025-08	1	0	0.0	NaN	
83	2025-09	5	3	265.0	88.333333	
84	2025-10	3	0	0.0	NaN	
85	2025-11	1	0	0.0	NaN	
86	2025-12	1	0	0.0	NaN	
87	2026-01	4	2	126.0	63.000000	
88	2026-02	4	1	338.0	338.000000	
89	2026-03	2	0	0.0	NaN	
90	2026-05	2	1	3.0	3.000000	
91	2026-06	5	2	65.0	32.500000	
92	2026-08	1	0	0.0	NaN	

	month	activity_count	score_record_count	total_engagement	average_engagement	med
93	2026-09	15	0	0.0		NaN

	spike_id	timestamp_time_window	activity_count	engagement_value	baseline	spike_ratio
0	ES1	2023-03	6	1704.0	6.0	
1	ES2	2026-02	4	338.0	6.0	
2	ES3	2025-09	5	265.0	6.0	





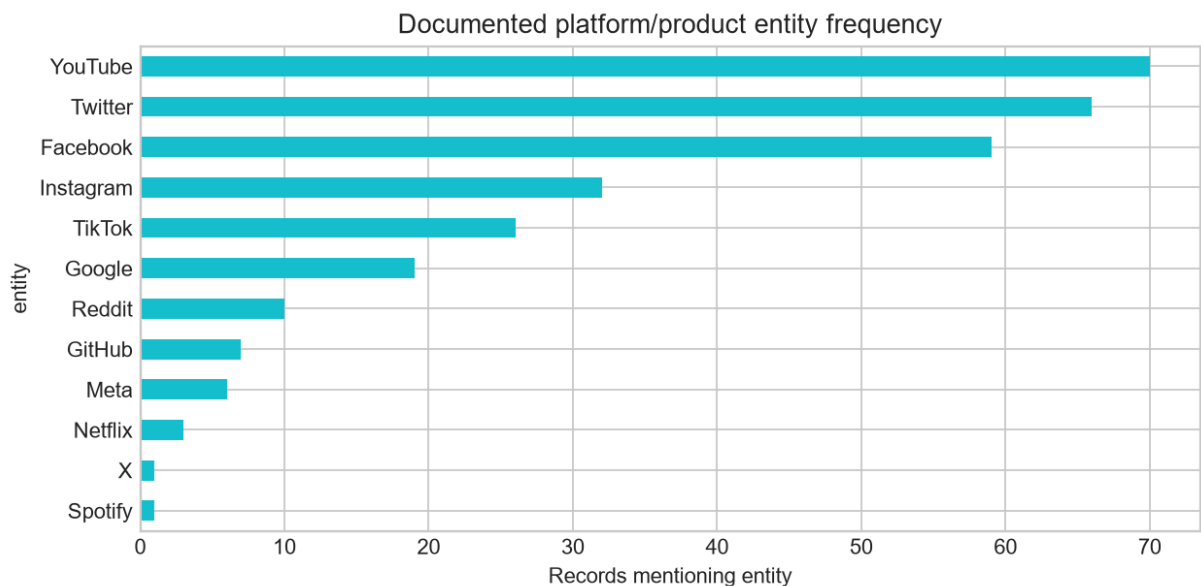
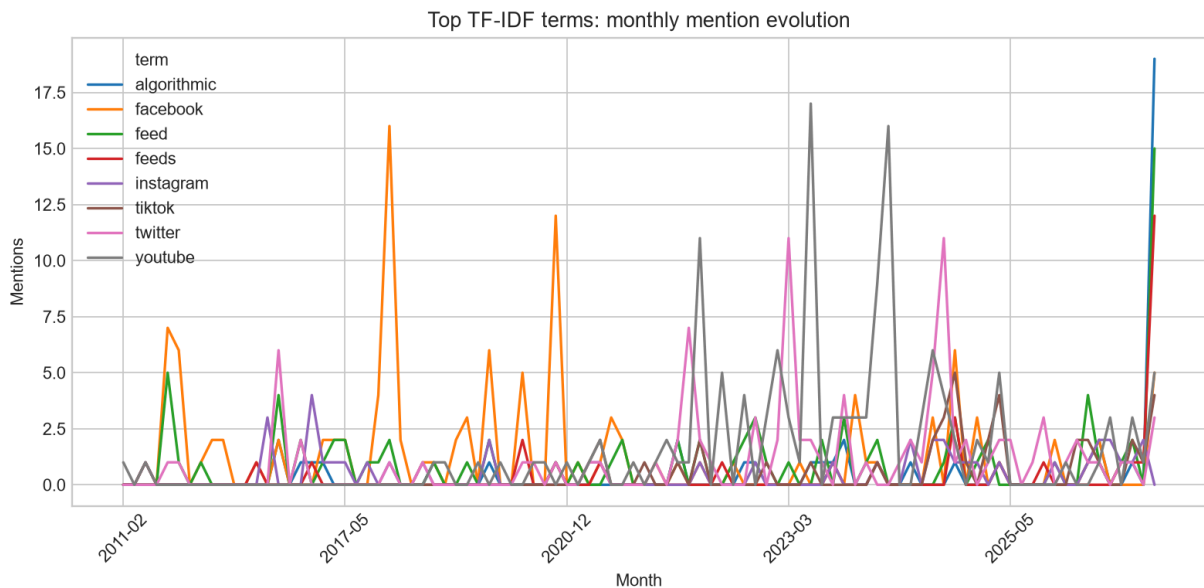
5. Data-derived TF-IDF terms and entities

TF-IDF uses English and documented custom stopwords. Terms are data-derived keywords, not semantic topic-model labels. Entities use a documented platform/product gazetteer; bare `X` is excluded unless explicit platform context is present.

```
In [6]: topics = pd.read_csv(ROOT/'results/topic_results.csv')
entities = pd.read_csv(ROOT/'results/entity_results.csv')
display(topics.head(15))
display(entities)
for f in ['topic_evolution.png', 'entity_frequency.png']:
    display(Image(filename=str(ROOT/'figures'/f)))
```

	term	tfidf_total
0	youtube	35.252315
1	twitter	26.910664
2	facebook	26.416824
3	feed	25.485057
4	algorithmic	17.977656
5	tiktok	12.112217
6	instagram	11.225886
7	feeds	11.112986
8	better	10.330280
9	source	10.252936
10	platform	10.169427
11	changes	9.906023
12	videos	9.029882
13	open	8.903374
14	platforms	8.898789

	entity	frequency	method
0	YouTube	70	case-insensitive gazetteer, documented platfor...
1	Twitter	66	case-insensitive gazetteer, documented platfor...
2	Facebook	59	case-insensitive gazetteer, documented platfor...
3	Instagram	32	case-insensitive gazetteer, documented platfor...
4	TikTok	26	case-insensitive gazetteer, documented platfor...
5	Google	19	case-insensitive gazetteer, documented platfor...
6	Reddit	10	case-insensitive gazetteer, documented platfor...
7	GitHub	7	case-insensitive gazetteer, documented platfor...
8	Meta	6	case-insensitive gazetteer, documented platfor...
9	Netflix	3	case-insensitive gazetteer, documented platfor...
10	Spotify	1	case-insensitive gazetteer, documented platfor...
11	X	1	explicit X/Twitter platform-context regex; bar...



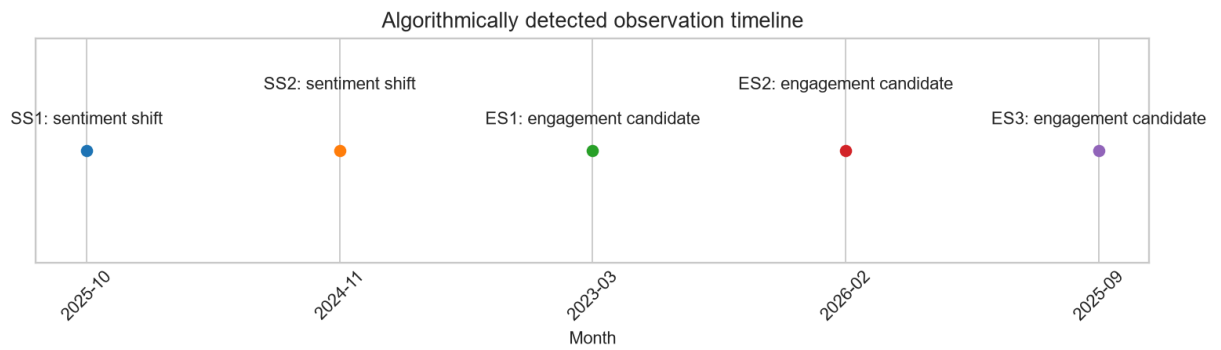
6. Trigger evidence, findings, limitations, and conclusion

Evidence records establish temporal coincidence only. No causal claim is made. The historical archive window, technical-community source, sparse monthly windows, and score-missing comments limit inference.

```
In [7]: evidence = pd.read_csv(ROOT/'results/trigger_evidence.csv')
display(evidence[['event_id','article_or_post_title','publication_timestamp','url',
display(Image(filename=str(ROOT/'figures/trigger_timeline.png'))))
print('Findings:')
print('- Final retained records:', len(df))
print('- Sentiment counts:', counts.to_dict())
print('- Descriptive candidate shifts:', shifts[['shift_id','start_time','end_time'])
print('- Highest available-score engagement candidate:', spikes.iloc[0][['timestamp',
print('Conclusion: Findings are reproducible, source-bound descriptive observations
```

event_id	article_or_post_title	publication_timestamp	url	
0	ES1 Twitter's Recommendation Algorithm	2023-03-31 18:42:16+00:00	https://news.ycombinator.com/item?id=35391433	cc
1	ES1 Twitter reveals some of its source code, inclu...	2023-03-31 22:02:37+00:00	https://news.ycombinator.com/item?id=35394275	cc
2	ES1 Twitter will open source all code used to reco...	2023-03-17 22:38:00+00:00	https://news.ycombinator.com/item?id=35203828	cc
3	SS1 NL Judge: Meta must respect user's choice of r...	2025-10-02 18:28:07+00:00	https://news.ycombinator.com/item?id=45453468	cc
4	SS1 Study finds growing social circles may fuel po...	2025-10-28 08:41:36+00:00	https://news.ycombinator.com/item?id=45730455	cc
5	SS1 Grokikipedia Review	2025-10-29 04:16:25+00:00	https://news.ycombinator.com/item?id=45742557	cc
6	SS2 Trudeau government bans TikTok from operating ...	2024-11-08 17:42:11+00:00	https://news.ycombinator.com/item?id=42088781	cc
7	SS2 Maybe Bluesky has "won"	2024-11-15 20:18:42+00:00	https://news.ycombinator.com/item?id=42150481	cc
8	SS2 Ask HN: Where to advertise my SaaS other than ...	2024-11-26 11:51:02+00:00	https://news.ycombinator.com/item?id=42244890	cc





Findings:

- Final retained records: 204
- Sentiment counts: {'Positive': 34, 'Negative': 132, 'Neutral': 38}
- Descriptive candidate shifts: [{'shift_id': 'SS1', 'start_time': '2025-09', 'end_time': '2025-10', 'magnitude': -0.8}, {'shift_id': 'SS2', 'start_time': '2024-10', 'end_time': '2024-11', 'magnitude': 0.8}]
- Highest available-score engagement candidate: {'timestamp_time_window': '2023-03', 'engagement_value': 1704.0, 'baseline': 6.0}

Conclusion: Findings are reproducible, source-bound descriptive observations; they do not prove causal public reaction.